

EXPERIMENT NO. 4

Aim: Write a program in Solidity to implement transactions by embedding of wallet.

Theory

A smart contract is a program deployed on a blockchain that executes predefined operations when its functions are called.

In this experiment, a Solidity smart contract named SimpleWallet is created. The contract stores an owner address and provides functions to:

- Receive ETH
- Check the wallet balance
- Withdraw ETH
- Generate deposit and withdrawal events

The owner is assigned during contract deployment using:

```
owner = msg.sender;
```

Only the owner is allowed to withdraw funds from the embedded wallet.

Solidity Program

Create a file named SimpleWallet.sol in Remix IDE and write the following program:

```
// SPDX-License-Identifier: MIT
```

```
pragma solidity ^0.8.0;
```

```
contract SimpleWallet {
```

```
    address public owner;
```

```
    event Deposit(address indexed sender, uint amount);
```

```
    event Withdraw(address indexed recipient, uint amount);
```

```
    constructor() {
```

```

        owner = msg.sender;
    }

    receive() external payable {
        emit Deposit(msg.sender, msg.value);
    }

    function withdraw(uint _amount) public {
        require(msg.sender == owner, "Only owner can withdraw");

        payable(owner).transfer(_amount);

        emit Withdraw(owner, _amount);
    }

    function getBalance() public view returns (uint) {
        return address(this).balance;
    }
}

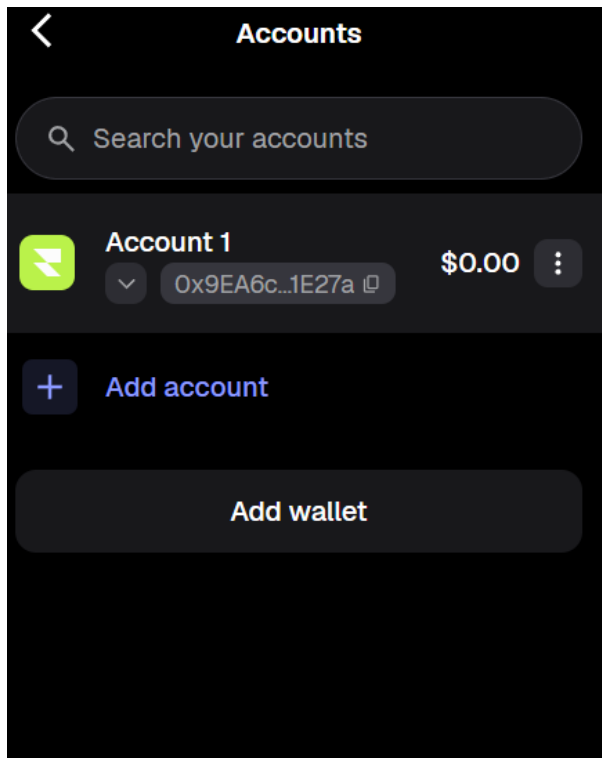
```

This is the same SimpleWallet structure given in the provided experiment guide.

Procedure

Step 1: Setup MetaMask Wallet

MetaMask was installed and an account was created. The wallet address was obtained for interacting with the Ethereum test network.

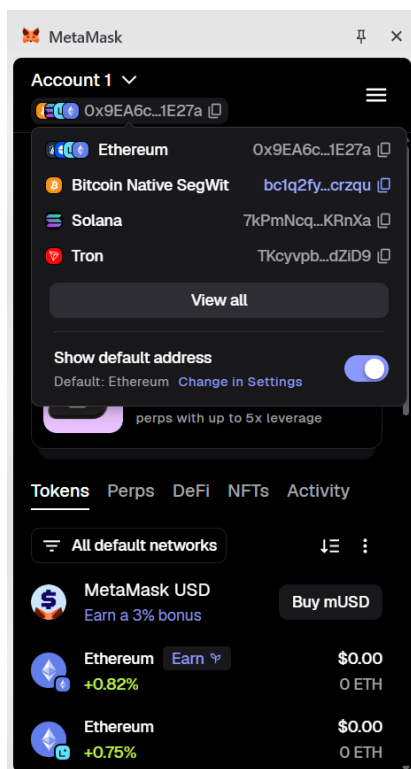


Step 2: Switch to Sepolia Testnet

MetaMask was switched from the Ethereum main network to the Ethereum Sepolia Testnet.

Sepolia is used because it provides test ETH for development and testing without using real ETH.

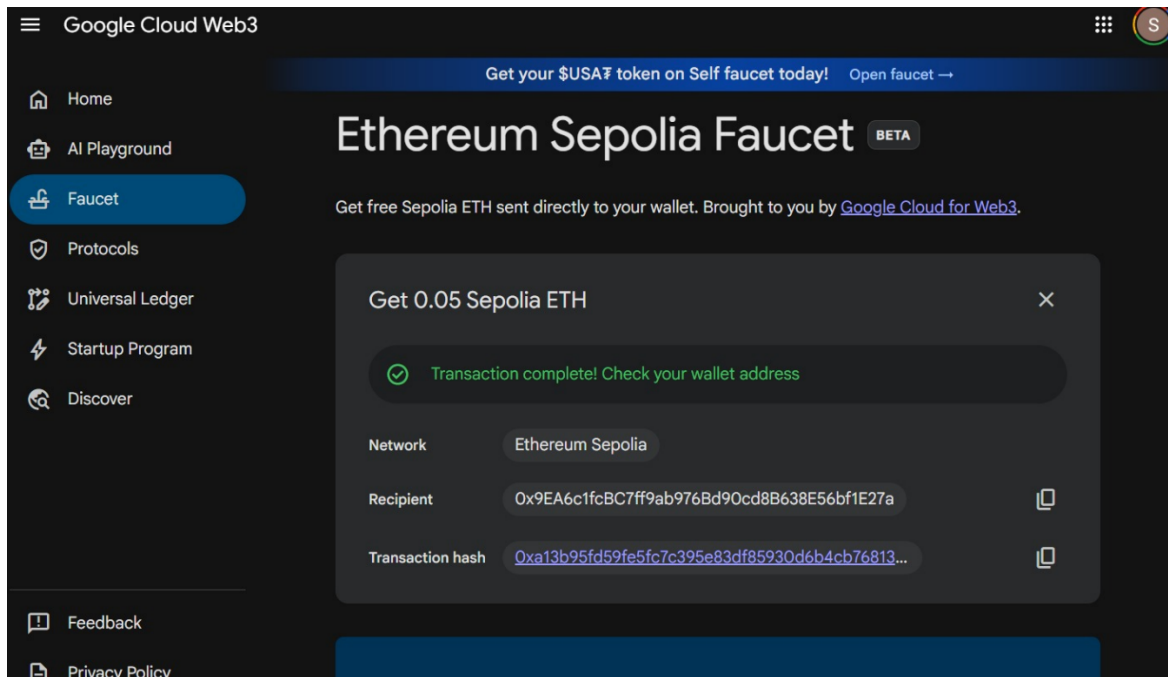
The provided guide also specifies switching MetaMask to the Sepolia Test Network.



Step 3: Obtain Sepolia Test ETH

Test ETH was obtained from a Sepolia faucet by providing the MetaMask wallet address.

The received ETH is test ETH and is used to pay transaction/deployment gas on the Sepolia network.

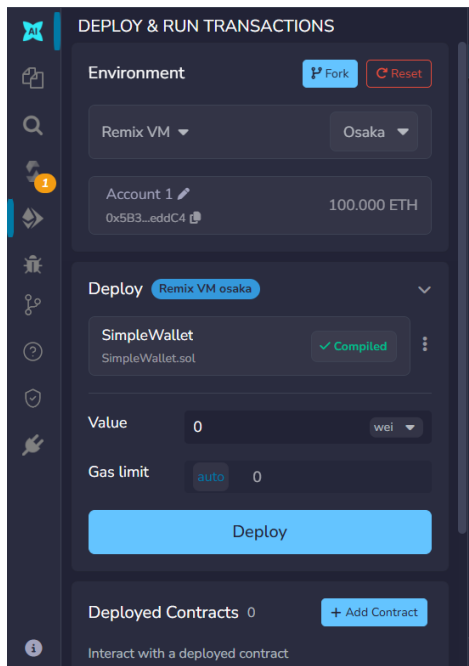


Step 4: Connect Remix IDE with MetaMask

Remix IDE was opened in the same browser as MetaMask.

In Deploy & Run Transactions, the environment was connected to MetaMask and the Sepolia network was selected.

The experiment guide specifies connecting Remix to MetaMask before deployment.

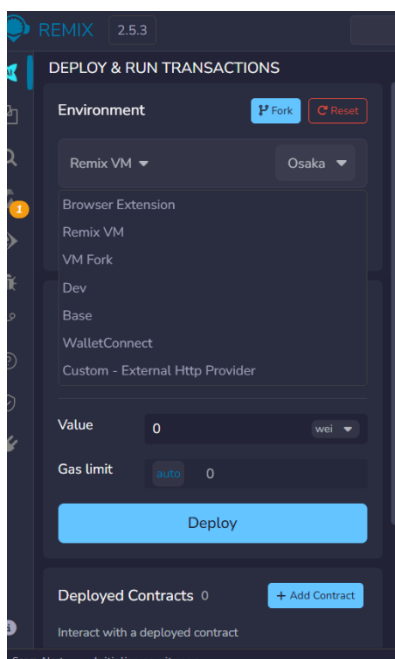


Step 5: Compile the Smart Contract

The Solidity Compiler tab was opened in Remix.

The appropriate Solidity compiler version was selected and SimpleWallet.sol was compiled successfully.

The guide specifies selecting a compiler version $\geq 0.8.0$ and compiling the contract.



Step 6: Deploy the Smart Contract

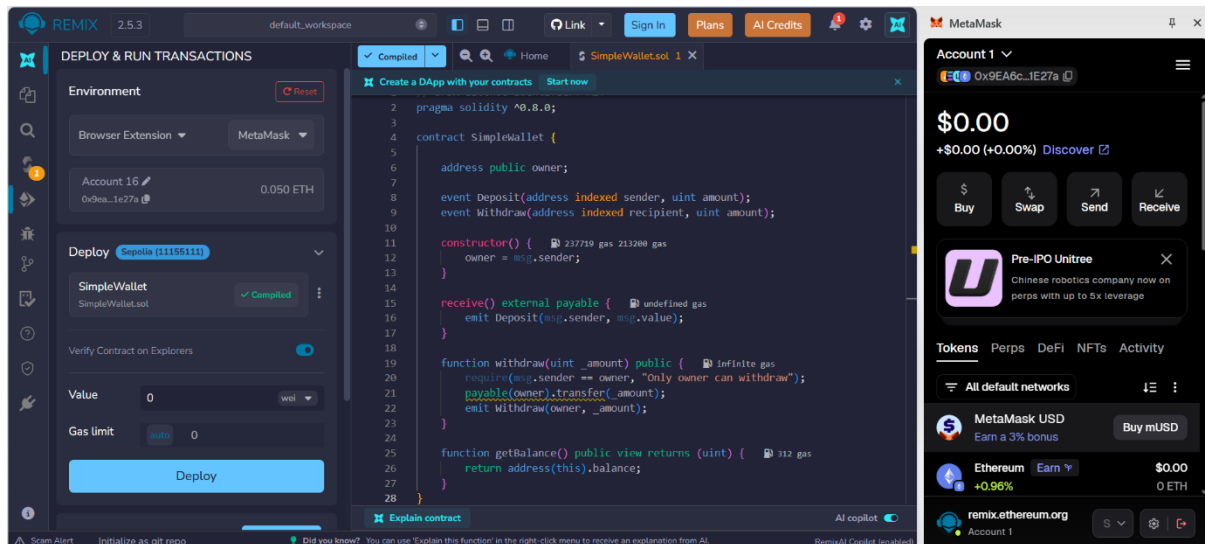
The Deploy & Run Transactions tab was opened.

The environment was connected to MetaMask and the Sepolia network was selected.

The SimpleWallet contract was deployed.

MetaMask requested confirmation for the gas fee, and the transaction was confirmed.

The provided guide specifies deploying the contract on Sepolia and copying the deployed contract address.



Step 7: Send ETH to the Embedded Wallet

After deployment, the deployed SimpleWallet contract address was obtained.

Using MetaMask, 0.001 ETH was transferred to the deployed SimpleWallet contract.

The image is a screenshot of the MetaMask 'Transactions' page. It shows a list of three transactions. The first transaction is a 'Transfer' of 0.001 ETH from 0x9EA6c1fc...56bf1E27a to 0x0FC5BE67...7716D2Aab. The second transaction is a 'Contract Creation' of 0 ETH from 0x9EA6c1fc...56bf1E27a to a new contract address. The third transaction is a 'Transfer' of 0.05 ETH from 0x52f1984C...2E7aCEedD to 0x9EA6c1fc...56bf1E27a.

Transaction Hash	Method	Block	Age	From	To	Amount	Txn Fee
0x9910c53fb67...	Transfer	11520722	18 mins ago	0x9EA6c1fc...56bf1E27a	0x0FC5BE67...7716D2Aab	0.001 ETH	0.00005768
0x6bb46a104b...	Contract Creation	11520692	24 mins ago	0x9EA6c1fc...56bf1E27a	Contract Creation	0 ETH	0.00079161
0xa13b95fd59f...	Transfer	11520588	45 mins ago	0x52f1984C...2E7aCEedD	0x9EA6c1fc...56bf1E27a	0.05 ETH	0.00002244

Step 8: Check Contract Balance

The deployed SimpleWallet contract was opened in Remix.

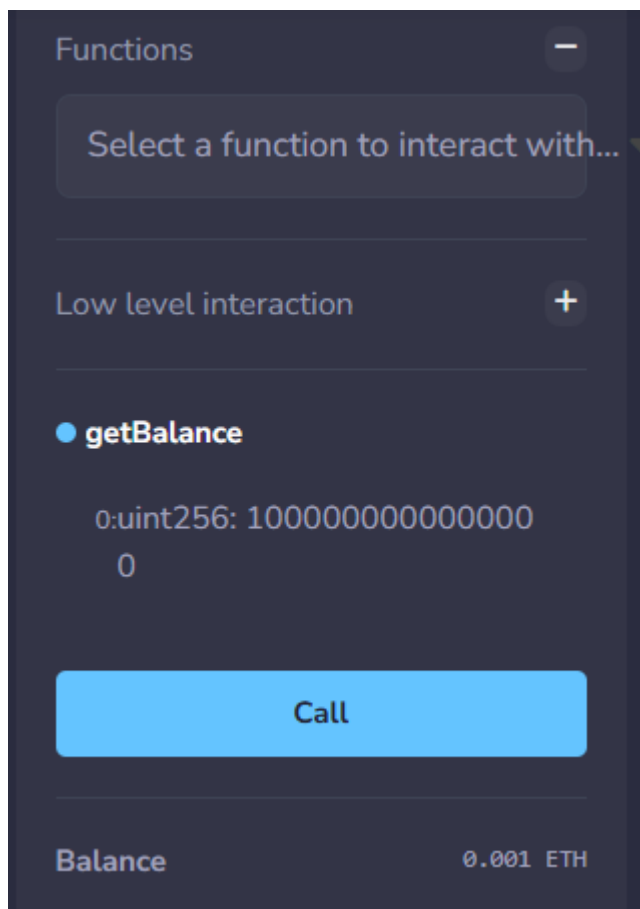
The getBalance() function was called to check the ETH stored in the contract.

The result was:

10000000000000000 wei

This is equivalent to:

0.001 ETH



Step 9: Withdraw ETH from the Embedded Wallet

The withdraw() function was selected.

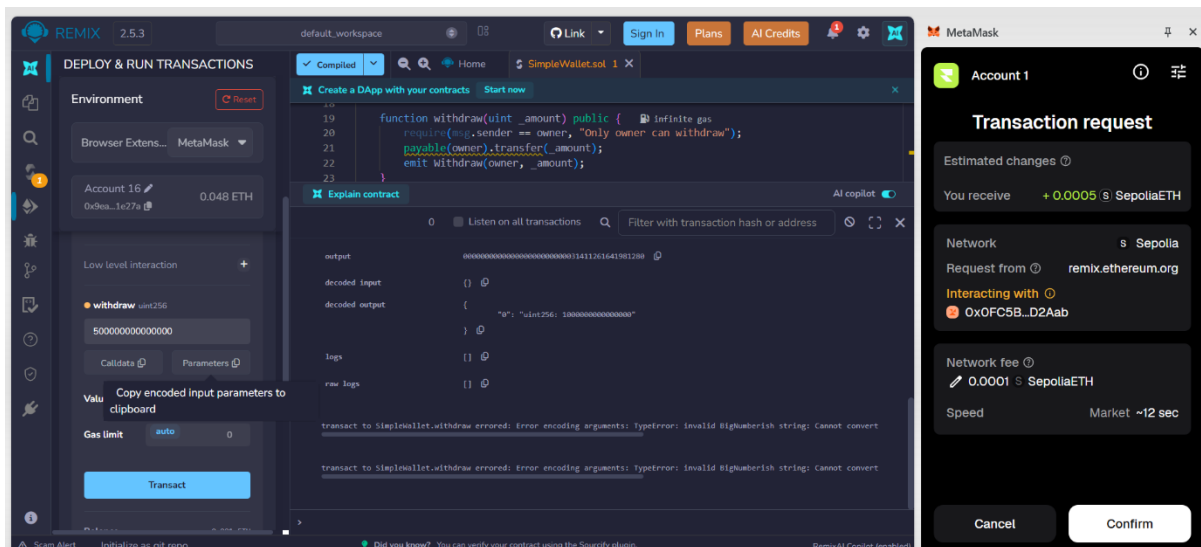
The withdrawal amount was entered in wei:

5000000000000000

This corresponds to:

0.0005 ETH

The transaction was submitted through MetaMask and successfully confirmed.



Step 10: Verify Remaining Balance

After the withdrawal, `getBalance()` was called again.

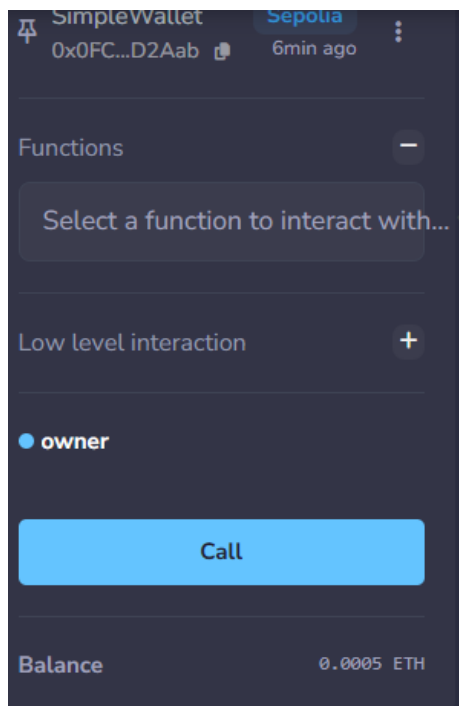
The result was:

5000000000000000 wei

which is equivalent to:

0.0005 ETH

Thus, the withdrawal transaction was successfully executed.



Result

The SimpleWallet smart contract was successfully implemented and deployed on the Ethereum Sepolia Testnet using Solidity and Remix IDE.

A transaction of 0.001 ETH was sent to the embedded wallet. The `getBalance()` function successfully displayed 0.001 ETH. Subsequently, 0.0005 ETH was withdrawn using the `withdraw()` function, and the final contract balance was verified as 0.0005 ETH.

Thus, transactions using an embedded wallet were successfully implemented and verified on the Sepolia test network.

Conclusion

The experiment demonstrated how a Solidity smart contract can function as an embedded wallet. The contract was able to receive ETH, maintain its balance, and allow the owner to withdraw funds. Remix IDE and MetaMask were used to deploy and interact with the contract on the Ethereum Sepolia Testnet.